

i am gonna be participating in a cyber competition. we are required to defend our windows server that has users connected and services such as email server. we should secure the server but still make sure it functions well like it would in a business. so there will be a red team that will try to hack into the server but the server should be functional and secure that they wont be able to get in. give me a checklist ranked by priority of what needs to be secured first and leave anything out. after the checklist give s



⊠ Thought for 3 seconds >

Securing a Windows Server for a cyber competition while keeping it functional for users and an email service is a critical balancing act. The key is to prioritize actions that block the most common attack vectors first, without breaking essential business services like email.

Here is a comprehensive checklist ranked by priority, followed by step-by-step instructions for each item.

Part 1: Priority-Ranked Hardening Checklist

This checklist is ordered from highest to lowest priority. Items at the top address the most immediate, high-impact vulnerabilities and should be completed first.

Priority 1: Immediate Threat Mitigation (Do These First)

- 1. Ensure All Critical Windows Updates Are Installed.** Unpatched vulnerabilities are the primary entry point for attackers. Install all available security and critical updates immediately.
- 2. Harden Local User Accounts.** Remove or disable unauthorized accounts, rename the default Administrator account, and ensure the Guest account is disabled. Attackers frequently target default and weak accounts.
- 3. Enforce Strong Password and Account Lockout Policies.** Configure a robust password policy (length, complexity, history) and an account lockout policy to block

brute-force attacks.

4. **Enable and Configure Windows Defender Firewall.** Activate the firewall for all profiles (Domain, Private, Public) and set the default inbound action to "Block." Only open ports that are absolutely necessary for business services (e.g., SMTP, HTTPS for email).

Priority 2: Core System and Service Hardening

5. **Disable Unnecessary Services, Roles, and Features.** Reduce the attack surface by removing anything not required for the server's role (e.g., print spooler, SMBv1, legacy protocols like LLMNR and NetBIOS).
6. **Harden the Email Server (e.g., Exchange or SMTP).** Restrict administrative access, enforce TLS encryption, enable MFA for admin accounts, and ensure the server is fully patched.
7. **Configure Advanced Audit Policies.** Enable detailed logging for critical events (logon, account management, object access) so you can detect and investigate suspicious activity.
8. **Secure Remote Management.** Disable RDP where possible, or if it must be enabled, restrict it to specific IP addresses and enforce Network Level Authentication (NLA). Use WinRM over HTTPS for remote PowerShell.

Priority 3: Advanced Hardening and Monitoring

9. **Implement Application Control (AppLocker or WDAC).** Create rules to allow only approved applications to run. This prevents attackers from executing malicious tools.
10. **Harden Authentication Protocols.** Disable NTLMv1 and LM, require SMB and LDAP signing, and prefer Kerberos authentication.
11. **Configure Windows Defender Antivirus.** Ensure real-time protection, cloud-delivered protection, and attack surface reduction (ASR) rules are enabled.
12. **Secure the Boot Process.** Enable Secure Boot and TPM, and consider enabling Credential Guard to protect credential hashes.
13. **Review and Harden Group Policy Objects (GPOs).** Import and apply security baselines (e.g., from Microsoft Security Compliance Toolkit) and remove any legacy or

conflicting settings.

14. **Restrict and Monitor Scheduled Tasks and Services.** Audit for any unusual scheduled tasks or services that could be used for persistence.
15. **Enable PowerShell Logging and Constrained Language Mode.** This helps detect and block malicious PowerShell scripts.

Part 2: Step-by-Step Instructions

Priority 1: Immediate Threat Mitigation

1. Ensure All Critical Windows Updates Are Installed

1. Open **Server Manager**.
2. Click on **Local Server** in the left pane.
3. In the **Properties** area, locate **Windows Update**.
4. Click on **Check for updates** to open the Windows Update settings.
5. Click **Check for updates** and install all critical, security, and recommended updates ³⁸ .
6. Reboot the server if prompted.

2. Harden Local User Accounts

1. Open **Computer Management** (press `Win + X` and select **Computer Management**).
2. Navigate to **Local Users and Groups > Users**.
3. **Remove Unauthorized Users:** Compare the list of accounts against the competition's README or authorized user list. Right-click any unauthorized account and select **Delete** ³⁸ .
4. **Disable the Guest Account:** Right-click the **Guest** account and select **Properties**. Check **Account is disabled** ³⁸ .
5. **Rename the Administrator Account:** Right-click the **Administrator** account and select **Rename**. Change it to a non-obvious name to make automated attacks harder ⁴⁸ .
6. **Check for Hidden/Backdoor Accounts:** Open PowerShell and run
`Get-LocalUser | Select-Object Name, Enabled, Description` . Look for any suspicious

accounts, especially those ending with a \$ (usually system accounts, but could be backdoors) ³⁸ .

3. Enforce Strong Password and Account Lockout Policies

1. Open **Local Security Policy** (press Win + R , type `secpol.msc` , and press Enter).
2. Navigate to **Account Policies > Password Policy**. Configure the following:
 - **Enforce password history**: 24 or more passwords ¹¹ .
 - **Maximum password age**: 365 or fewer days (but not 0) ¹¹ .
 - **Minimum password length**: 14 or more characters ¹¹ .
 - **Password must meet complexity requirements**: Enabled ¹¹ .
 - **Store passwords using reversible encryption**: Disabled ¹¹ .
3. Navigate to **Account Policies > Account Lockout Policy**. Configure the following:
 - **Account lockout threshold**: 5 invalid logon attempts.
 - **Account lockout duration**: 30 minutes.
 - **Reset account lockout counter after**: 30 minutes ³⁸ .

4. Enable and Configure Windows Defender Firewall

1. Open **Server Manager**, click **Tools**, and select **Windows Defender Firewall with Advanced Security** ² .
2. In the console, right-click **Windows Defender Firewall with Advanced Security on Local Computer** and select **Properties**.
3. For each profile (**Domain**, **Private**, **Public**):
 - Set the **Firewall state** to **On**.
 - Set **Inbound connections** to **Block (default)**.
 - Set **Outbound connections** to **Allow (default)** ² .
4. Click **OK**.
5. To open a specific port (e.g., for email):
 - In the left pane, right-click **Inbound Rules** and select **New Rule**.
 - Select **Port**, click **Next**.
 - Enter the port number (e.g., 25 for SMTP, 443 for HTTPS), click **Next**.

- Select **Allow the connection**, click **Next**.
- Select when the rule applies (Domain, Private, Public), click **Next**.
- Give the rule a name (e.g., "SMTP Inbound") and click **Finish**.

Priority 2: Core System and Service Hardening

5. Disable Unnecessary Services, Roles, and Features

1. Remove Unnecessary Features (e.g., SMBv1):

- Open **Server Manager**.
- Click **Manage > Remove Roles and Features**.
- Click **Next** until you reach the **Features** page.
- **Clear the checkbox** for **SMB 1.0/CIFS File Sharing Support**  .
- Click **Next** and then **Remove**.

2. Disable Unnecessary Services (e.g., Print Spooler):

- Press **Win + R** , type `services.msc` , and press **Enter**.
- Locate **Print Spooler**.
- Double-click it, set **Startup type** to **Disabled**, and click **Stop**  .

3. Disable Legacy Network Protocols (LLMNR, NetBIOS):

- Open **Group Policy Management** (`gpmc.msc`).
- Create a new GPO or edit an existing one for the server.
- Navigate to **Computer Configuration > Policies > Administrative Templates > Network > DNS Client**.
- Enable **Turn off Multicast Name Resolution**.
- Navigate to **Computer Configuration > Policies > Windows Settings > Security Settings > Local Policies > Security Options**.
- Disable **Network security: LAN Manager authentication level** (set to **Send NTLMv2 response only. Refuse LM & NTLM**) ⁴⁸ .

6. Harden the Email Server (e.g., Exchange or SMTP)

1. **Restrict Administrative Access:** Ensure only authorized accounts can access the Exchange Admin Center (EAC) or SMTP administrative interfaces  .

2. **Enable Multifactor Authentication (MFA):** Require MFA for all administrative accounts accessing the email server [↗](#) .
3. **Enforce TLS Encryption:**
 - In **Exchange Admin Center**, navigate to **Servers > Servers** > select your server > **Edit**.
 - Go to **Transport** and ensure TLS 1.2 is enabled for both inbound and outbound connections [↗](#) .
4. **Ensure Emergency Mitigation Service is Enabled:** In Exchange, run the **Exchange Emergency Mitigation Service** to apply the latest Microsoft mitigations automatically [↗](#) .
5. **Apply Security Baselines:** Use the **Exchange Server Security Best Practices** guide from CISA to apply recommended baselines ²⁸ .
6. **Disable Remote PowerShell:** Restrict or disable remote PowerShell access for non-administrative users [↗](#) .

7. Configure Advanced Audit Policies

1. Open **Group Policy Management** (`gpmc.msc`).
2. Right-click your **Default Domain Policy** and select **Edit**.
3. Navigate to **Computer Configuration > Policies > Windows Settings > Security Settings > Advanced Audit Policy Configuration > Audit Policies** [↗](#) .
4. Enable the following key subcategories (set both **Success** and **Failure**):
 - **Account Logon:** Audit Credential Validation, Audit Kerberos Authentication Service.
 - **Account Management:** Audit User Account Management, Audit Security Group Management.
 - **Logon/Logoff:** Audit Logon, Audit Logoff, Audit Special Logon.
 - **Object Access:** Audit File System, Audit Registry.
 - **Policy Change:** Audit Audit Policy Change.
5. Run `gpupdate /force` to apply the policies.

8. Secure Remote Management

1. **Disable RDP if Not Needed:** In **Server Manager**, go to **Local Server > Remote Desktop**. Click **Disabled**.
2. **If RDP is Required:**
 - Go to **System Properties** (press Win + R , type sysdm.cpl , press Enter).
 - Click the **Remote** tab. Check **Allow remote connections to this computer** and **Require Network Level Authentication (NLA)**.
 - Use **Windows Defender Firewall with Advanced Security** to create an inbound rule for RDP (port 3389) that only allows connections from specific, trusted IP addresses.
3. **Use WinRM over HTTPS:** Configure WinRM to use HTTPS for remote PowerShell sessions instead of HTTP.

Priority 3: Advanced Hardening and Monitoring

9. Implement Application Control (AppLocker)

1. Open **Local Security Policy** (secpol.msc).
2. Navigate to **Application Control Policies > AppLocker**.
3. Right-click **Executable Rules** and select **Create Default Rules** to allow programs in Program Files and Windows .
4. Create a new rule to allow your specific business applications (e.g., email server executables).
5. Create a final rule to **Deny** all other executables.

10. Harden Authentication Protocols

1. **Disable NTLMv1 and LM:** In **Local Security Policy**, navigate to **Local Policies > Security Options**. Set **Network security: LAN Manager authentication level** to **Send NTLMv2 response only. Refuse LM & NTLM** ⁴⁸ .
2. **Require SMB Signing:** In **Local Security Policy**, go to **Local Policies > Security Options**. Enable **Microsoft network server: Digitally sign communications (always)** and **Microsoft network client: Digitally sign communications (always)**.
3. **Require LDAP Signing:** In **Local Security Policy**, go to **Local Policies > Security Options**. Set **Domain controller: LDAP server signing requirements** to **Require**

signing ²⁰ .

11. Configure Windows Defender Antivirus

1. Open **Windows Security** (search from Start menu).
2. Go to **Virus & threat protection > Manage settings**.
3. Ensure **Real-time protection**, **Cloud-delivered protection**, and **Automatic sample submission** are all **On**.
4. Go to **App & browser control > Exploit protection > Attack surface reduction**. Enable all available ASR rules, especially those that block executable content from email and Office macros ²⁰ .

12. Secure the Boot Process

1. **Enable Secure Boot and TPM:** This is typically configured in the server's BIOS/UEFI firmware. Reboot the server and enter the firmware setup (usually F2, F10, or Del). Enable **Secure Boot** and **TPM 2.0** ¹ .
2. **Enable Credential Guard:** In **Group Policy Management**, navigate to **Computer Configuration > Administrative Templates > System > Device Guard**. Enable **Turn On Virtualization Based Security** and set **Credential Guard Configuration** to **Enabled with UEFI lock** ²⁰ .

13. Review and Harden Group Policy Objects (GPOs)

1. Download the **Microsoft Security Compliance Toolkit** and import the recommended **Windows Server Security Baseline GPO**.
2. In **Group Policy Management**, link the baseline GPO to the Organizational Unit (OU) containing your server ²⁰ .
3. Review the settings in the baseline and test them in a non-production environment before applying to the live server.

14. Restrict and Monitor Scheduled Tasks and Services

1. Open **Task Scheduler** and review all scheduled tasks. Disable or delete any tasks that are not recognized or not required for business operations.
2. Open **Services** (`services.msc`) and review all services. Set any unnecessary service to **Disabled** and stop it.

15. Enable PowerShell Logging and Constrained Language Mode

1. In **Group Policy Management**, navigate to **Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell**.
2. Enable **Turn on Module Logging** and **Turn on Script Block Logging**.
3. Enable **Turn on PowerShell Transcription** and specify an output directory.
4. To enforce **Constrained Language Mode**, navigate to **Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell** and enable **Turn on PowerShell Constrained Language Mode**.

By following this prioritized checklist and the corresponding GUI steps, you will significantly harden your Windows Server against red team attacks while maintaining the functionality required for users and the email service. Remember to test all changes in a safe environment if possible, and always have a rollback plan.

 127 web pages

okay give me shortcut commands i can run in powershell to complete the things on the checklist to save time give all that are possible

 Thought for 4 seconds >

Here are the PowerShell commands corresponding to the prioritized checklist from the previous answer. Run all commands in an elevated (Administrator) PowerShell session. Items that cannot be fully automated via PowerShell are noted accordingly.

Priority 1: Immediate Threat Mitigation

1. Install Critical Windows Updates

PowerShell can trigger the update scan and installation using the PSWindowsUpdate module. If the module is not installed, you can install it from the PowerShell Gallery.

```
powershell
```

```
# Install the PSWindowsUpdate module (if not already installed)
Install-Module -Name PSWindowsUpdate -Force

# Import the module
Import-Module PSWindowsUpdate

# Scan for and install all available updates
Get-WindowsUpdate -Install -AcceptAll -IgnoreReboot
```

If you prefer to use the built-in Windows Update COM object without installing a module:

```
powershell

# Trigger a Windows Update scan (this starts the scan but does not install)
$UpdateSession = New-Object -ComObject Microsoft.Update.Session
$UpdateSearcher = $UpdateSession.CreateUpdateSearcher()
$SearchResult = $UpdateSearcher.Search("IsInstalled=0")
$SearchResult.Updates | Select-Object Title
```

For automated installation, the PSWindowsUpdate module is the recommended approach. You can also use `sconfig` (Server Configuration tool) from the command line for a menu-driven update process.

2. Harden Local User Accounts

Disable the Guest account:

```
powershell

Disable-LocalUser -Name "Guest"
```

Disable any other unauthorized accounts:

```
powershell

# List all local users first to identify unauthorized accounts
Get-LocalUser | Select-Object Name, Enabled, Description
```

```
# Disable a specific account  
Disable-LocalUser -Name "AccountName"
```

Remove an unauthorized account entirely:

```
powershell  
  
Remove-LocalUser -Name "AccountName"
```

Rename the default Administrator account:

```
powershell  
  
Rename-LocalUser -Name "Administrator" -NewName "NewAdminName"
```

Check for hidden or suspicious accounts:

```
powershell  
  
Get-LocalUser | Select-Object Name, Enabled, Description, LastLogon
```

3. Enforce Strong Password and Account Lockout Policies

PowerShell can enforce these via the `net accounts` command or by setting local security policy through `secedit`.

Using net accounts (quick and effective):

```
powershell  
  
# Set minimum password length to 14 characters  
net accounts /minpwlen:14  
  
# Set maximum password age to 365 days  
net accounts /maxpwage:365  
  
# Set minimum password age to 1 day  
net accounts /minpwage:1  
  
# Set password history to 24 passwords
```

```
net accounts /uniquepw:24

# Set account lockout threshold to 5 invalid attempts
net accounts /lockoutthreshold:5

# Set Lockout duration to 30 minutes
net accounts /lockoutduration:30

# Set Lockout observation window to 30 minutes
net accounts /lockoutwindow:30
```

Enable password complexity (must be done via secedit or local security policy):

```
powershell

# Export the current security policy
secedit /export /cfg C:\secpol.cfg

# Edit the file to set PasswordComplexity = 1, then re-import
# (You can use PowerShell to modify the file)
(Get-Content C:\secpol.cfg) -replace "PasswordComplexity = 0", "PasswordComplexity = 1" |
Set-Content C:\secpol.cfg

# Apply the modified policy
secedit /configure /db secedit.sdb /cfg C:\secpol.cfg /areas SECURITYPOLICY
```

4. Enable and Configure Windows Defender Firewall

Enable the firewall for all profiles:

```
powershell

Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled True
```

Set default inbound action to Block and outbound to Allow:

```
powershell

Set-NetFirewallProfile -DefaultInboundAction Block -DefaultOutboundAction Allow -NotifyOn
Listen False -AllowUnicastResponseToMulticast True -LogFileName %SystemRoot%\System32\Log
Files\Firewall\pfirewall.log
```

Verify the firewall state:

```
powershell
```

```
Get-NetFirewallProfile | Select-Object Name, Enabled, DefaultInboundAction, DefaultOutboundAction
```

Open a specific port (example: SMTP port 25):

```
powershell
```

```
New-NetFirewallRule -DisplayName "SMTP Inbound" -Direction Inbound -Protocol TCP -LocalPort 25 -Action Allow
```

Open HTTPS port 443:

```
powershell
```

```
New-NetFirewallRule -DisplayName "HTTPS Inbound" -Direction Inbound -Protocol TCP -LocalPort 443 -Action Allow
```

Priority 2: Core System and Service Hardening

5. Disable Unnecessary Services, Roles, and Features

Disable SMBv1:

```
powershell
```

```
Set-SmbServerConfiguration -EnableSMB1Protocol $false -Confirm:$false
```

Or remove it entirely via DISM:

```
powershell
```

```
Disable-WindowsOptionalFeature -Online -FeatureName SMB1Protocol -NoRestart
```

Disable the Print Spooler service:

```
powershell
```

```
Stop-Service -Name Spooler -Force  
Set-Service -Name Spooler -StartupType Disabled
```

Disable LLMNR (Link-Local Multicast Name Resolution):

```
powershell
```

```
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient" -Force  
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient" -Name "EnableMulticast" -Value 0 -Type DWord
```

Disable NetBIOS over TCP/IP on all adapters:

```
powershell
```

```
$adapters = Get-WmiObject Win32_NetworkAdapterConfiguration -Filter "IPEnabled = True"  
foreach ($adapter in $adapters) {  
    $adapter.SetTcpipNetbios(2) # 2 = Disable NetBIOS over TCP/IP  
}
```

Disable PowerShell v2 (legacy version):

```
powershell
```

```
Disable-WindowsOptionalFeature -Online -FeatureName MicrosoftWindowsPowerShellV2 -NoRestart
```

6. Harden the Email Server (Exchange)

If running on-premises Exchange, these commands require the Exchange Management Shell.

Enforce TLS on the transport service:

```
powershell
```

```
Set-TransportService -Identity "YourMailServerName" -RequireTLS $true
```

Disable Basic Authentication for SMTP AUTH:

```
powershell
```

```
Set-TransportConfig -SmtpClientAuthenticationDisabled $true
```

Enable auditing on all mailboxes:

```
powershell
```

```
Get-Mailbox -ResultSize Unlimited | Set-Mailbox -AuditEnabled $true -AuditAdmin Update,Copy,Move,MoveToDeletedItems,SoftDelete,HardDelete,SendAs,SendOnBehalf -AuditDelegate Update,SoftDelete,HardDelete,SendAs,SendOnBehalf -AuditOwner Update,SoftDelete,HardDelete,Move,MoveToDeletedItems
```

Restrict remote PowerShell access (Exchange 2019/2016):

```
powershell
```

```
Set-User -Identity "UserName" -RemotePowerShellEnabled $false
```

To disable remote PowerShell for all non-admin users:

```
powershell
```

```
Get-User -ResultSize Unlimited | Where-Object {$_.RemotePowerShellEnabled -eq $true} | Set-User -RemotePowerShellEnabled $false
```

Enable the Exchange Emergency Mitigation Service:

```
powershell
```

```
Set-ExchangeServer -Identity "YourServerName" -MitigationsEnabled $true
```

For MFA on the email server, you typically need to integrate with Azure AD or use a third-party MFA solution. This cannot be fully configured via a single PowerShell command.

7. Configure Advanced Audit Policies

Use `auditpol` (available from PowerShell) to enable specific audit subcategories.

Enable Logon/Logoff auditing:

powershell

```
auditpol /set /subcategory:"Logon" /success:enable /failure:enable  
auditpol /set /subcategory:"Logoff" /success:enable  
auditpol /set /subcategory:"Special Logon" /success:enable
```

Enable Account Logon auditing:

powershell

```
auditpol /set /subcategory:"Credential Validation" /success:enable /failure:enable  
auditpol /set /subcategory:"Kerberos Authentication Service" /success:enable /failure:enable
```

Enable Account Management auditing:

powershell

```
auditpol /set /subcategory:"User Account Management" /success:enable /failure:enable  
auditpol /set /subcategory:"Security Group Management" /success:enable /failure:enable
```

Enable Object Access auditing:

powershell

```
auditpol /set /subcategory:"File System" /success:enable /failure:enable  
auditpol /set /subcategory:"Registry" /success:enable /failure:enable
```

Enable Policy Change auditing:

powershell

```
auditpol /set /subcategory:"Audit Policy Change" /success:enable /failure:enable
```

Verify all audit policies:

powershell

```
auditpol /get /category:*
```

8. Secure Remote Management

Disable RDP (if not required):

```
powershell
```

```
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server" -Name "fDenyTSConnections" -Value 1
```

If RDP is required, restrict it to specific IP addresses via firewall:

```
powershell
```

```
# First remove the default RDP allow rule
```

```
Remove-NetFirewallRule -DisplayName "Remote Desktop - User Mode (TCP-In)" -ErrorAction SilentlyContinue
```

```
# Then create a restricted rule for a specific IP
```

```
New-NetFirewallRule -DisplayName "RDP Restricted" -Direction Inbound -Protocol TCP -LocalPort 3389 -RemoteAddress "192.168.1.100" -Action Allow
```

Stop and disable WinRM (if remote PowerShell is not needed):

```
powershell
```

```
Stop-Service -Name WinRM -Force
```

```
Set-Service -Name WinRM -StartupType Disabled
```

Disable the WinRM firewall rules:

```
powershell
```

```
Set-NetFirewallRule -DisplayName "Windows Remote Management (HTTP-In)" -Enabled False
```

Priority 3: Advanced Hardening and Monitoring

9. Implement Application Control (AppLocker)

Check if AppLocker is available:

```
powershell
```

```
Get-WindowsFeature -Name Application-Server
```

Install AppLocker feature if needed:

```
powershell
```

```
Install-WindowsFeature -Name Application-Server -IncludeManagementTools
```

Import the AppLocker module and generate default rules:

```
powershell
```

```
Import-Module AppLocker  
$defaultRules = Get-AppLockerPolicy -Local -Xml  
Set-AppLockerPolicy -XmlPolicy $defaultRules -Merge
```

AppLocker policy creation is most reliably done through Group Policy or the Local Security Policy MMC snap-in. PowerShell is primarily used for importing, exporting, and testing policies.

10. Harden Authentication Protocols

Disable NTLMv1 and LM (force NTLMv2 only):

```
powershell
```

```
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa" -Name "LmCompatibilityLevel" -Value 5 -Type DWord
```

Require SMB signing on the server:

```
powershell
```

```
Set-SmbServerConfiguration -RequireSecuritySignature $true -EnableSecuritySignature $true  
-Confirm:$false
```

Require SMB signing on the client:

powershell

```
Set-SmbClientConfiguration -RequireSecuritySignature $true -EnableSecuritySignature $true  
-Confirm:$false
```

Disable NTLM secrets for SMB (Windows Server 2025 and Windows 11 24H2+):

powershell

```
Set-SmbClientConfiguration -DisableNTLM $true
```

Require LDAP signing (domain controllers only):

powershell

```
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\NTDS\Parameters" -Name "L  
DAPServerIntegrity" -Value 2 -Type DWord
```

11. Configure Windows Defender Antivirus

Enable real-time protection:

powershell

```
Set-MpPreference -DisableRealtimeMonitoring $false
```

Enable cloud-delivered protection:

powershell

```
Set-MpPreference -MAPSReporting Advanced  
Set-MpPreference -SubmitSamplesConsent SendSafeSamples
```

Enable ASR rules (set to Block mode):

```
powershell
```

```
# Block abuse of exploited vulnerable signed drivers
```

```
Add-MpPreference -AttackSurfaceReductionRules_Ids "56A863A9-875E-4185-98A7-B882C64B5CE5"  
-AttackSurfaceReductionRules_Actions "Enabled"
```

```
# Block credential stealing from lsass.exe
```

```
Add-MpPreference -AttackSurfaceReductionRules_Ids "9E6C4E1F-7D60-472F-BA1A-A39EF669E4B2"  
-AttackSurfaceReductionRules_Actions "Enabled"
```

```
# Block execution of potentially obfuscated scripts
```

```
Add-MpPreference -AttackSurfaceReductionRules_Ids "5BEB7EFE-FD9A-4556-801D-275E5FFC04CC"  
-AttackSurfaceReductionRules_Actions "Enabled"
```

```
# Block persistence through WMI event subscription
```

```
Add-MpPreference -AttackSurfaceReductionRules_Ids "E6DB77E5-3DF2-4CF1-B95A-636979351E5B"  
-AttackSurfaceReductionRules_Actions "Enabled"
```

```
# Use advanced protection against ransomware
```

```
Add-MpPreference -AttackSurfaceReductionRules_Ids "C1DB55AB-C21A-4637-BB3F-A12568109D35"  
-AttackSurfaceReductionRules_Actions "Enabled"
```

Verify ASR rule status:

```
powershell
```

```
Get-MpPreference | Select-Object AttackSurfaceReductionRules_Ids, AttackSurfaceReductionR  
ules_Actions
```

12. Secure the Boot Process

Enable Virtualization-Based Security (VBS) for Credential Guard:

```
powershell
```

```
# Enable VBS
```

```
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard" -Name "Enable  
VirtualizationBasedSecurity" -Value 1 -Type DWord
```

```
# Require Secure Boot (value 1) or Secure Boot + DMA (value 3)
```

```
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard" -Name "Requir  
ePlatformSecurityFeatures" -Value 1 -Type DWord
```

```
# Enable Credential Guard with UEFI Lock (value 1) or without Lock (value 2)
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa" -Name "LsaCfgFlags" -
Value 1 -Type DWord
```

Verify Credential Guard status:

```
powershell
```

```
Get-CimInstance -ClassName Win32_DeviceGuard -Namespace root\Microsoft\Windows\DeviceGuard
```

Secure Boot and TPM must be enabled in the server's BIOS/UEFI firmware. This requires a reboot and manual entry into the firmware setup utility.

13. Review and Harden Group Policy Objects

PowerShell can import and apply GPOs if you have the Microsoft Security Compliance Toolkit.

Import the Windows Server Security Baseline:

```
powershell
```

```
# Download the toolkit first, then run:
Import-GPO -BackupGpoName "MSFT Windows Server 2022 - Domain Security" -Path "C:\Baseline
s\Windows Server 2022 Security Baseline" -TargetName "ServerBaseline" -CreateIfNeeded
```

This requires the Group Policy module and administrative access to Active Directory.

14. Restrict and Monitor Scheduled Tasks and Services

Disable a specific scheduled task:

```
powershell
```

```
Disable-ScheduledTask -TaskName "TaskName"
```

Disable all tasks in a specific folder (example: Windows Update):

```
powershell
```

```
Get-ScheduledTask -TaskPath "\Microsoft\Windows\WindowsUpdate\" | Disable-ScheduledTask
```

List all scheduled tasks for review:

```
powershell
```

```
Get-ScheduledTask | Select-Object TaskName, TaskPath, State
```

Disable an unnecessary service:

```
powershell
```

```
Stop-Service -Name "ServiceName" -Force  
Set-Service -Name "ServiceName" -StartupType Disabled
```

List all running services for review:

```
powershell
```

```
Get-Service | Where-Object {$_.Status -eq "Running"} | Select-Object Name, DisplayName, StartType
```

15. Enable PowerShell Logging and Constrained Language Mode

Enable Script Block Logging (via registry):

```
powershell
```

```
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -Force  
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -Name "EnableScriptBlockLogging" -Value 1 -Type DWord
```

Enable Module Logging (via registry):

```
powershell
```

```
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging" -Force
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging" -Name "EnableModuleLogging" -Value 1 -Type DWord

New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging\ModuleNames" -Force
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging\ModuleNames" -Name "*" -Value "*" -Type String
```

Enable Transcription:

powershell

```
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" -Force
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" -Name "EnableTranscripting" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" -Name "OutputDirectory" -Value "C:\PSLogs" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" -Name "EnableInvocationHeader" -Value 1 -Type DWord

# Create the Log directory
New-Item -Path "C:\PSLogs" -ItemType Directory -Force
```

Enable Constrained Language Mode (via environment variable):

powershell

```
[Environment]::SetEnvironmentVariable('__PSLockdownPolicy', '4', 'Machine')
```

Verify logging is enabled:

powershell

```
Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -ErrorAction SilentlyContinue
Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" -ErrorAction SilentlyContinue
```

Summary of Items Requiring Manual Configuration

The following items cannot be fully automated via PowerShell alone and require manual GUI or firmware-level configuration:

1. **Secure Boot and TPM** – Must be enabled in the server's BIOS/UEFI firmware.
2. **MFA for Email Server** – Requires integration with Azure AD or a third-party MFA provider.
3. **Application Control (AppLocker)** – Policy creation is most reliably done via Group Policy MMC or Local Security Policy MMC.
4. **Group Policy Object Import** – Requires the Group Policy Management Console and appropriate AD permissions.
5. **Windows Updates** – While PSWindowsUpdate can automate installation, some updates may require manual approval or a reboot outside of business hours.



159 web pages

can you right a script that will secure everything at once, i sthat possible

Thought for 31 seconds >

Yes, it is possible. Below is a single consolidated script that walks the entire checklist in priority order, logs every action, backs up the current configuration before changing anything, and continues on failure instead of stopping halfway.

Read the caveats at the end before you run it. A one-shot hardening script can absolutely lock you out or break the mail service if you run it blind.

The Script

Save this as `Invoke-ServerHardening.ps1` .

```
powershell
```

```
#Requires -RunAsAdministrator
```

```
<#
```

.SYNOPSIS

Priority-ordered Windows Server hardening for a cyber competition.

.DESCRIPTION

Applies Priority 1, 2 and 3 hardening in a single pass. Backs up the current configuration first, logs every action, and continues past non-fatal failures.

.PARAMETER DryRun

Preview only. Logs what would be done without changing anything.

.PARAMETER PreserveRDP

Do not disable RDP. Auto-enabled if you are connected over RDP.

.PARAMETER PreserveWinRM

Do not stop or disable the WinRM service.

.PARAMETER SkipUpdates

Skip the Windows Update phase (slow, may trigger reboots).

.PARAMETER PreserveFirewall

Do not modify firewall profiles or rules.

.PARAMETER SkipRenameAdmin

Do not rename the built-in Administrator account.

.PARAMETER EnableConstrainedLanguage

Enable PowerShell Constrained Language Mode. Requires reboot. Can break admin scripts and some business applications. Off by default.

.PARAMETER LogRoot

Directory for logs and configuration backups. Default C:\HardeningLogs

.EXAMPLE

```
.\Invoke-ServerHardening.ps1 -DryRun
```

.EXAMPLE

```
.\Invoke-ServerHardening.ps1 -PreserveRDP -PreserveWinRM
```

#>

```
[CmdletBinding()]
```

```
param(
```

```
    [switch]$DryRun,
```

```
    [switch]$PreserveRDP,
```

```
    [switch]$PreserveWinRM,
```

```
    [switch]$SkipUpdates,
```

```
    [switch]$PreserveFirewall,
```

```
    [switch]$SkipRenameAdmin,
```

```
    [switch]$EnableConstrainedLanguage,
```

```
    [string]$LogRoot = 'C:\HardeningLogs'
```

```
)
```

```
# -----
```

```
# Setup
```

```
# -----
```

```
$stamp = Get-Date -Format 'yyyyMMdd_HH:mm:ss'
```

```

$script:LogFile = Join-Path $LogRoot "Hardening_$stamp.log"
$script:BackupRoot= Join-Path $LogRoot "Backup_$stamp"
$script:Failures = New-Object System.Collections.Generic.List[string]
$script:RebootNeeded = $false

if (-not $DryRun) {
    New-Item -Path $LogRoot -ItemType Directory -Force | Out-Null
    New-Item -Path $script:BackupRoot -ItemType Directory -Force | Out-Null
}

function Write-Log {
    param([string]$Message, [ValidateSet('INFO','OK','WARN','ERROR','HEAD')][string]$Level = 'INFO')
    $line = "[{0}] [{1}] {2}" -f (Get-Date -Format 'yyyy-MM-dd HH:mm:ss'), $Level, $Message
    if (-not $DryRun) { Add-Content -Path $script:LogFile -Value $line -ErrorAction SilentlyContinue }
    switch ($Level) {
        'ERROR' { Write-Host $line -ForegroundColor Red }
        'WARN' { Write-Host $line -ForegroundColor Yellow }
        'OK' { Write-Host $line -ForegroundColor Green }
        'HEAD' { Write-Host $line -ForegroundColor Cyan }
        default { Write-Host $line }
    }
}

function Invoke-Step {
    param(
        [Parameter(Mandatory)][string]$Name,
        [Parameter(Mandatory)][scriptblock]$Action
    )
    if ($DryRun) { Write-Log "DRYRUN: $Name" 'WARN'; return }
    try {
        & $Action | Out-Null
        Write-Log "OK: $Name" 'OK'
    } catch {
        Write-Log "FAIL: $Name -- $($_.Exception.Message)" 'ERROR'
        $script:Failures.Add("$Name :: $($_.Exception.Message)")
    }
}

function Set-RegValue {
    param(
        [Parameter(Mandatory)][string]$Path,

```

```

[Parameter(Mandatory)][string]$Name,
[Parameter(Mandatory)]$Value,
[ValidateSet('DWord','String','QWord','MultiString','ExpandString')][string]$Type
= 'DWord'
)
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name $Name -Value $Value -Type $Type -Force
}

# -----
# Preflight
# -----

Write-Log "=== Windows Server Hardening Started ===" 'HEAD'
Write-Log "Host: $env:COMPUTERNAME | User: $env:USERNAME | Session: $env:SESSIONNAME"
Write-Log "DryRun=$DryRun PreserveRDP=$PreserveRDP PreserveWinRM=$PreserveWinRM SkipUpdates=$SkipUpdates PreserveFirewall=$PreserveFirewall"
Write-Log "Log: $script:LogFile"
Write-Log "Backups: $script:BackupRoot"

if ($env:SESSIONNAME -like 'RDP*' -and -not $PreserveRDP) {
    Write-Log "Detected an RDP session and -PreserveRDP was not supplied. Auto-enabling t
o prevent lockout." 'WARN'
    $PreserveRDP = $true
}

# -----
# Phase 0: Backup current state
# -----

Write-Log "--- Phase 0: Backup and Restore Point ---" 'HEAD'

Invoke-Step "Backup: local security policy" {
    secedit /export /cfg "$script:BackupRoot\secpol_before.inf" | Out-Null
    if ($LASTEXITCODE -ne 0) { throw "secedit export failed ($LASTEXITCODE)" }
}
Invoke-Step "Backup: firewall rules" {
    Get-NetFirewallRule | Export-Clixml "$script:BackupRoot\firewall_before.xml"
}
Invoke-Step "Backup: local users" {
    Get-LocalUser | Export-Clixml "$script:BackupRoot\localusers_before.xml"
}
Invoke-Step "Backup: services" {
    Get-Service | Select-Object Name,DisplayName,Status,StartType |
        Export-Csv "$script:BackupRoot\services_before.csv" -NoTypeInformation
}

```

```

Invoke-Step "Backup: Defender preferences" {
    Get-MpPreference | Export-Clixml "$script:BackupRoot\mppref_before.xml"
}
Invoke-Step "Backup: account policy" {
    net accounts | Out-File "$script:BackupRoot\accounts_before.txt"
}
Invoke-Step "Backup: scheduled tasks" {
    Get-ScheduledTask | Select-Object TaskName,TaskPath,State |
        Export-Csv "$script:BackupRoot\scheduledtasks_before.csv" -NoTypeInfoation
}
Invoke-Step "Create system restore point" {
    Enable-ComputerRestore -Drive "$env:SystemDrive\" -ErrorAction SilentlyContinue
    Checkpoint-Computer -Description "Pre-Hardening" -RestorePointType MODIFY_SETTINGS -E
rrorAction Stop
}

# -----
# Phase 1: Windows Updates
# -----
if (-not $SkipUpdates) {
    Write-Log "--- Phase 1: Windows Updates ---" 'HEAD'

    Invoke-Step "Ensure PSWindowsUpdate module is present" {
        if (-not (Get-Module -ListAvailable -Name PSWindowsUpdate)) {
            Install-Module PSWindowsUpdate -Force -Confirm:$false -Scope AllUsers
        }
    }
    Invoke-Step "Install all available updates" {
        Import-Module PSWindowsUpdate -Force
        Get-WindowsUpdate -Install -AcceptAll -IgnoreReboot -ErrorAction Stop
        $script:RebootNeeded = $true
    }
} else {
    Write-Log "Phase 1 skipped by request" 'WARN'
}

# -----
# Phase 2: Local accounts
# -----
Write-Log "--- Phase 2: Local Accounts ---" 'HEAD'

Invoke-Step "Disable Guest account" {
    $g = Get-LocalUser -Name 'Guest' -ErrorAction SilentlyContinue
    if ($g -and $g.Enabled) { Disable-LocalUser -Name 'Guest' }
}

```

```

}

if (-not $SkipRenameAdmin) {
    Invoke-Step "Rename built-in Administrator account" {
        $a = Get-LocalUser -SID 'S-1-5-21-*-500' -ErrorAction SilentlyContinue
        if (-not $a) { $a = Get-LocalUser -Name 'Administrator' -ErrorAction SilentlyContinue }
        if ($a -and $a.Name -eq 'Administrator') {
            Rename-LocalUser -Name 'Administrator' -NewName 'svc-localadmin'
            Write-Log "Built-in Administrator renamed to svc-localadmin" 'WARN'
        }
    }
}

Invoke-Step "Inventory local accounts for manual review" {
    Get-LocalUser | Select-Object Name,Enabled,LastLogon,Description |
        Export-Csv "$script:BackupRoot\localusers_after.csv" -NoTypeInformation
    Get-LocalUser | Where-Object Enabled |
        ForEach-Object { Write-Log "    Enabled local account: $($_.Name)" }
}

# -----
# Phase 3: Password and lockout policy
# -----
Write-Log "--- Phase 3: Password and Lockout Policy ---" 'HEAD'

Invoke-Step "Apply account and lockout policy" {
    net accounts /minpwlen:14 /maxpwage:365 /minpwage:1 /uniquepw:24 `
        /lockoutthreshold:5 /lockoutduration:30 /lockoutwindow:30 | Out-Null
    if ($LASTEXITCODE -ne 0) { throw "net accounts failed with exit code $LASTEXITCODE" }
}

Invoke-Step "Enable password complexity requirement" {
    $cfg = Join-Path $env:TEMP "secpol_hardening_$stamp.inf"
    secedit /export /cfg $cfg | Out-Null
    $content = Get-Content $cfg
    if ($content -match 'PasswordComplexity\s*=\s*0') {
        $content -replace 'PasswordComplexity\s*=\s*0', 'PasswordComplexity = 1' | Set-Content $cfg
    } elseif (-not ($content -match 'PasswordComplexity')) {
        $content += 'PasswordComplexity = 1'
        $content | Set-Content $cfg
    }
    secedit /configure /db "$env:TEMP\secedit_$stamp.sdb" /cfg $cfg /areas SECURITYPOLICY
}

```

```

| Out-Null
    if ($LASTEXITCODE -ne 0) { throw "secdit configure failed ($LASTEXITCODE)" }
    Remove-Item $cfg -Force -ErrorAction SilentlyContinue
}

# -----
# Phase 4: Firewall
# -----

if (-not $PreserveFirewall) {
    Write-Log "--- Phase 4: Windows Defender Firewall ---" 'HEAD'

    Invoke-Step "Enable firewall on all profiles" {
        Set-NetFirewallProfile -Profile Domain,Private,Public -Enabled True
    }
    Invoke-Step "Default inbound block, outbound allow" {
        Set-NetFirewallProfile -Profile Domain,Private,Public `
            -DefaultInboundAction Block -DefaultOutboundAction Allow
    }
    Invoke-Step "Enable firewall logging" {
        Set-NetFirewallProfile -Profile Domain,Private,Public `
            -LogAllowed True -LogBlocked True -LogMaxSizeKilobytes 16384 `
            -LogFileName '%SystemRoot%\System32\LogFiles\Firewall\pfirewall.log'
    }
    Invoke-Step "Verify firewall state" {
        Get-NetFirewallProfile | Select-Object Name,Enabled,DefaultInboundAction |
            Format-Table | Out-String | ForEach-Object { Write-Log $_ }
    }
} else {
    Write-Log "Phase 4 skipped by request" 'WARN'
}

# -----
# Phase 5: Services, roles and features
# -----

Write-Log "--- Phase 5: Attack Surface Reduction (Services and Features) ---" 'HEAD'

Invoke-Step "Disable SMBv1 server protocol" {
    Set-SmbServerConfiguration -EnableSMB1Protocol $false -Force
}
Invoke-Step "Remove SMBv1 optional feature" {
    $f = Get-WindowsOptionalFeature -Online -FeatureName SMB1Protocol -ErrorAction SilentlyContinue
    if ($f -and $f.State -eq 'Enabled') {
        Disable-WindowsOptionalFeature -Online -FeatureName SMB1Protocol -NoRestart | Out
    }
}

```

```

-Null
    }
}
Invoke-Step "Disable Print Spooler service" {
    Stop-Service -Name Spooler -Force -ErrorAction SilentlyContinue
    Set-Service -Name Spooler -StartupType Disabled
}
Invoke-Step "Disable LLMNR" {
    Set-RegValue -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient' `
        -Name 'EnableMulticast' -Value 0
}
Invoke-Step "Disable NetBIOS over TCP/IP on all adapters" {
    Get-CimInstance Win32_NetworkAdapterConfiguration -Filter 'IPEnabled=True' |
        ForEach-Object {
            Invoke-CimMethod -InputObject $_ -MethodName SetTcpipNetbios `
                -Arguments @{ TcpipNetbiosOptions = 2 } | Out-Null
        }
}
Invoke-Step "Disable PowerShell v2 engine" {
    $f = Get-WindowsOptionalFeature -Online -FeatureName MicrosoftWindowsPowerShellV2Root
    -ErrorAction SilentlyContinue
    if ($f -and $f.State -eq 'Enabled') {
        Disable-WindowsOptionalFeature -Online -FeatureName MicrosoftWindowsPowerShellV2R
oot -NoRestart | Out-Null
    }
}
Invoke-Step "Disable SMB compression and insecure guest auth" {
    Set-SmbClientConfiguration -EnableInsecureGuestLogons $false -Force -ErrorAction Sile
ntlyContinue
}

# -----
# Phase 6: Email server (Exchange) hardening
# -----
if (Get-Command Get-TransportService -ErrorAction SilentlyContinue) {
    Write-Log "--- Phase 6: Exchange Server Hardening ---" 'HEAD'

    Invoke-Step "Require TLS on transport service" {
        Set-TransportService -Identity $env:COMPUTERNAME -RequireTLS $true
    }
    Invoke-Step "Disable SMTP client authentication" {
        Set-TransportConfig -SmtpClientAuthenticationDisabled $true
    }
    Invoke-Step "Enable mailbox auditing on all mailboxes" {

```

```

    Get-Mailbox -ResultSize Unlimited |
        Set-Mailbox -AuditEnabled $true `
            -AuditAdmin Update,Copy,Move,MoveToDeletedItems,SoftDelete,HardDelete,SendAs,SendOnBehalf `
            -AuditDelegate Update,SoftDelete,HardDelete,SendAs,SendOnBehalf `
            -AuditOwner Update,SoftDelete,HardDelete,Move,MoveToDeletedItems
    }
    Invoke-Step "Disable remote PowerShell for non-current users" {
        Get-User -ResultSize Unlimited |
            Where-Object { $_.RemotePowerShellEnabled -eq $true -and $_.Name -ne $env:USERNAME } |
            Set-User -RemotePowerShellEnabled $false
    }
    Invoke-Step "Enable Exchange Emergency Mitigation Service" {
        Set-ExchangeServer -Identity $env:COMPUTERNAME -MitigationsEnabled $true
    }
} else {
    Write-Log "Phase 6 skipped: Exchange cmdlets not present" 'WARN'
}

# -----
# Phase 7: Advanced auditing
# -----
Write-Log "--- Phase 7: Advanced Audit Policy ---" 'HEAD'

$auditMap = @{
    'Credential Validation'          = @('enable','enable')
    'Kerberos Authentication Service' = @('enable','enable')
    'Kerberos Service Ticket Operations' = @('enable','enable')
    'User Account Management'        = @('enable','enable')
    'Security Group Management'       = @('enable','enable')
    'Logon'                          = @('enable','enable')
    'Logoff'                         = @('enable','disable')
    'Special Logon'                  = @('enable','disable')
    'Account Lockout'                = @('enable','enable')
    'File System'                    = @('enable','enable')
    'Registry'                       = @('enable','enable')
    'Process Creation'               = @('enable','disable')
    'Audit Policy Change'            = @('enable','enable')
    'Security State Change'          = @('enable','enable')
}

foreach ($sub in $auditMap.Keys) {
    $s = $auditMap[$sub][0]

```

```

    $f = $auditMap[$sub][1]
    Invoke-Step "Audit policy: $sub" {
        auditpol /set /subcategory:"$sub" /success:$s /failure:$f | Out-Null
        if ($LASTEXITCODE -ne 0) { throw "auditpol failed for $sub" }
    }
}

Invoke-Step "Include command line in process creation events" {
    Set-RegValue -Path 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit' `
        -Name 'ProcessCreationIncludeCmdLine_Enabled' -Value 1
}

Invoke-Step "Expand audit log size to 192 MB" {
    auditpol /set /category:* /option:192 | Out-Null
}

# -----
# Phase 8: Remote management
# -----
Write-Log "--- Phase 8: Remote Management ---" 'HEAD'

if (-not $PreserveRDP) {
    Invoke-Step "Disable RDP" {
        Set-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server' `
            -Name 'fDenyTSConnections' -Value 1 -Type DWord
        Disable-NetFirewallRule -DisplayGroup 'Remote Desktop' -ErrorAction SilentlyContinue
    }
} else {
    Invoke-Step "Harden RDP and keep it enabled" {
        Set-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server' `
            -Name 'fDenyTSConnections' -Value 0 -Type DWord
        Set-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp' `
            -Name 'UserAuthentication' -Value 1 -Type DWord
        Set-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp' `
            -Name 'SecurityLayer' -Value 2 -Type DWord
    }
}

if (-not $PreserveWinRM) {
    Invoke-Step "Stop and disable WinRM" {
        Stop-Service -Name WinRM -Force -ErrorAction SilentlyContinue
    }
}

```

```

Set-Service -Name WinRM -StartupType Disabled
Disable-NetFirewallRule -DisplayGroup 'Windows Remote Management' -ErrorAction Si
lentlyContinue
}
}

Invoke-Step "Disable Remote Assistance" {
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Remote Assistance' `
        -Name 'fAllowToGetHelp' -Value 0
}

# -----
# Phase 9: Authentication protocols
# -----
Write-Log "--- Phase 9: Authentication Protocol Hardening ---" 'HEAD'

Invoke-Step "Force NTLMv2 only (refuse LM and NTLMv1)" {
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa' `
        -Name 'LmCompatibilityLevel' -Value 5
}

Invoke-Step "Do not store LM hash" {
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa' -Name 'NoLMHash' -Val
ue 1
}

Invoke-Step "Disable WDigest credential caching" {
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest'
    ,
        -Name 'UseLogonCredential' -Value 0
}

Invoke-Step "Require SMB server signing" {
    Set-SmbServerConfiguration -RequireSecuritySignature $true -EnableSecuritySignature
    $true -Force
}

Invoke-Step "Require SMB client signing" {
    Set-SmbClientConfiguration -RequireSecuritySignature $true -EnableSecuritySignature
    $true -Force
}

Invoke-Step "Restrict anonymous SID and share enumeration" {
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa' -Name 'RestrictAnonym
ous' -Value 1
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa' -Name 'RestrictAnonym
ousSAM' -Value 1
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa' -Name 'EveryoneInclud
esAnonymous' -Value 0
}

```

```
}  
Invoke-Step "Require LDAP signing on domain controller" {  
    if (Get-Service -Name NTDS -ErrorAction SilentlyContinue) {  
        Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Services\NTDS\Parameters' `   
            -Name 'LDAPServerIntegrity' -Value 2  
    }  
}  
  
# -----  
# Phase 10: Windows Defender  
# -----  
Write-Log "--- Phase 10: Windows Defender ---" 'HEAD'  
  
Invoke-Step "Enable real-time protection" {  
    Set-MpPreference -DisableRealtimeMonitoring $false  
}  
Invoke-Step "Enable cloud-delivered protection" {  
    Set-MpPreference -MAPSReporting Advanced -SubmitSamplesConsent SendSafeSamples  
}  
Invoke-Step "Enable network protection" {  
    Set-MpPreference -EnableNetworkProtection Enabled  
}  
Invoke-Step "Enable PUA protection" {  
    Set-MpPreference -PUAProtection Enabled  
}  
Invoke-Step "Enable behavior monitoring and scanning of network files" {  
    Set-MpPreference -DisableBehaviorMonitoring $false  
    Set-MpPreference -DisableScanningNetworkFiles $false  
    Set-MpPreference -DisableScriptScanning $false  
}  
Invoke-Step "Update Defender signatures" {  
    Update-MpSignature -ErrorAction Stop  
}  
  
$asrRules = [ordered]@{  
    '56A863A9-875E-4185-98A7-B882C64B5CE5' = 'Block abuse of exploited vulnerable signed  
drivers'  
    '9E6C4E1F-7D60-472F-BA1A-A39EF669E4B2' = 'Block credential stealing from lsass.exe'  
    '5BEB7EFE-FD9A-4556-801D-275E5FFC04CC' = 'Block execution of potentially obfuscated s  
cripts'  
    'E6DB77E5-3DF2-4CF1-B95A-636979351E5B' = 'Block persistence through WMI event subscri  
ption'  
    'C1DB55AB-C21A-4637-BB3F-A12568109D35' = 'Use advanced protection against ransomware'  
    'D3E037E1-3EB8-44C8-A917-57927947596D' = 'Block executable content from email and web
```

```

mail'
    'BE9BA2D9-53EA-4CDC-84E5-9B1EEEE46550' = 'Block executables unless they meet prevalen
ce or trust'
    '01443614-CD74-433A-B99E-2ECDC07BFC25' = 'Block untrusted and unsigned processes from
USB'
    'B2B3F03D-6A65-4F7B-A9C7-1C7EF74A9BA4' = 'Block untrusted and unsigned processes from
Office macros'
    '92E97FA1-2EDF-4476-BDD6-9DD0B4DDDC7B' = 'Block Office apps from creating child proce
sses'
    '3B576869-A4EC-4529-8536-B80A7769E899' = 'Block Office apps from creating executable
content'
    '26190899-1602-49E8-8B27-EB1D0A1CE869' = 'Block Office communication app from creatin
g child processes'
}

foreach ($id in $asrRules.Keys) {
    $desc = $asrRules[$id]
    Invoke-Step "ASR rule: $desc" {
        Add-MpPreference -AttackSurfaceReductionRules_Ids $id `
            -AttackSurfaceReductionRules_Actions Enabled
    }
}

Invoke-Step "Verify ASR rules applied" {
    $p = Get-MpPreference
    for ($i = 0; $i -lt $p.AttackSurfaceReductionRules_Ids.Count; $i++) {
        Write-Log (" ASR {0} = {1}" -f $p.AttackSurfaceReductionRules_Ids[$i],
            $p.AttackSurfaceReductionRules_Actions[$i])
    }
}

# -----
# Phase 11: Boot and credential protection
# -----
Write-Log "--- Phase 11: Boot and Credential Protection ---" 'HEAD'

Invoke-Step "Enable Virtualization Based Security" {
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard' `
        -Name 'EnableVirtualizationBasedSecurity' -Value 1
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard' `
        -Name 'RequirePlatformSecurityFeatures' -Value 1
}

Invoke-Step "Enable Credential Guard with UEFI lock" {
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa' -Name 'LsaCfgFlags' -

```

```

Value 1
}
Invoke-Step "Enable Hypervisor Enforced Code Integrity" {
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity' `
        -Name 'Enabled' -Value 1
}
Invoke-Step "Enable LSA protection (RunAsPPL)" {
    Set-RegValue -Path 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa' -Name 'RunAsPPL' -Value 1
}
$script:RebootNeeded = $true

# -----
# Phase 12: Scheduled task and service review
# -----
Write-Log "--- Phase 12: Scheduled Task and Service Review ---" 'HEAD'

Invoke-Step "Inventory scheduled tasks after hardening" {
    Get-ScheduledTask | Select-Object TaskName,TaskPath,State |
        Export-Csv "$script:BackupRoot\scheduledtasks_after.csv" -NoTypeInfo
}
Invoke-Step "List non-Microsoft scheduled tasks for manual review" {
    Get-ScheduledTask |
        Where-Object { $_.TaskPath -notlike '\Microsoft\*' -and $_.State -ne 'Disabled' }
    |
        ForEach-Object { Write-Log " Task: $($_.TaskPath)$($_.TaskName)" }
}
Invoke-Step "List auto-start services for manual review" {
    Get-Service | Where-Object { $_.StartType -eq 'Automatic' } |
        ForEach-Object { Write-Log " Service: $($_.Name) [$($_.Status)]" }
}

# -----
# Phase 13: PowerShell Logging
# -----
Write-Log "--- Phase 13: PowerShell Logging ---" 'HEAD'

Invoke-Step "Enable script block logging" {
    Set-RegValue -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging' `
        -Name 'EnableScriptBlockLogging' -Value 1
}
Invoke-Step "Enable module logging" {

```

```

Set-RegValue -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLoggin
g' `
    -Name 'EnableModuleLogging' -Value 1
Set-RegValue -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLoggin
g\ModuleNames' `
    -Name '*' -Value '*' -Type String
}
Invoke-Step "Enable transcription" {
    New-Item -Path 'C:\PSLogs' -ItemType Directory -Force | Out-Null
    Set-RegValue -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcriptio
n' `
        -Name 'EnableTranscripting' -Value 1
    Set-RegValue -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcriptio
n' `
        -Name 'OutputDirectory' -Value 'C:\PSLogs' -Type String
    Set-RegValue -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcriptio
n' `
        -Name 'EnableInvocationHeader' -Value 1
}
Invoke-Step "Enable PowerShell v7 script block logging" {
    Set-RegValue -Path 'HKLM:\SOFTWARE\Policies\Microsoft\PowerShellCore\ScriptBlockLoggi
ng' `
        -Name 'EnableScriptBlockLogging' -Value 1
}

if ($EnableConstrainedLanguage) {
    Invoke-Step "Enable PowerShell Constrained Language Mode" {
        [Environment]::SetEnvironmentVariable('__PSLockdownPolicy', '4', 'Machine')
        $script:RebootNeeded = $true
    }
} else {
    Write-Log "Constrained Language Mode NOT enabled. Pass -EnableConstrainedLanguage to
turn it on." 'WARN'
}

# -----
# Finalise
# -----
Write-Log "--- Finalisation ---" 'HEAD'

Invoke-Step "Refresh group policy" {
    gpupdate /force /wait:60 | Out-Null
}

```

```
Write-Log "=== Hardening Complete ===" 'HEAD'
Write-Log "Log file:    $script:LogFile"
Write-Log "Backups:     $script:BackupRoot"

if ($script:Failures.Count -gt 0) {
    Write-Log "$($script:Failures.Count) step(s) failed:" 'ERROR'
    $script:Failures | ForEach-Object { Write-Log " $_" 'ERROR' }
} else {
    Write-Log "All steps completed without error." 'OK'
}

if ($script:RebootNeeded) {
    Write-Log "A REBOOT IS REQUIRED to activate VBS, Credential Guard, LSA protection and
update changes." 'WARN'
    Write-Log "Reboot manually when ready. Do not reboot mid-competition without a plan."
'WARN'
}

Write-Log "Reminder: Secure Boot and TPM must still be enabled in BIOS/UEFI manually." 'W
ARN'
Write-Log "Reminder: MFA on the mail service must still be configured through Azure AD or
your MFA provider." 'WARN'
```

How to Run It

Always preview first:

```
powershell
```

```
Set-ExecutionPolicy Bypass -Scope Process -Force
.\Invoke-ServerHardening.ps1 -DryRun
```

Then run for real, keeping RDP and WinRM alive so you do not cut your own connection:

```
powershell
```

```
.\Invoke-ServerHardening.ps1 -PreserveRDP -PreserveWinRM
```

If you are on the physical console or a hypervisor console and want the strictest posture:

```
powershell
```

```
.\Invoke-ServerHardening.ps1
```

If you already patched and do not want the update phase:

```
powershell
```

```
.\Invoke-ServerHardening.ps1 -SkipUpdates -PreserveRDP -PreserveWinRM
```

The script auto-detects an RDP session and turns on `-PreserveRDP` for you if you forget. That guard has saved more competitions than any hardening rule.

What the Script Cannot Do

These remain manual, and skipping them leaves gaps a red team will find:

1. **Secure Boot and TPM** must be enabled in BIOS/UEFI firmware.
2. **MFA on the mail service** requires Azure AD, Duo, or another MFA provider.
3. **AppLocker or WDAC policy authoring** is far more reliable through the Group Policy or Local Security Policy MMC. The script does not attempt to guess which applications your business needs.
4. **Group Policy baseline import** from the Microsoft Security Compliance Toolkit needs the Group Policy Management Console and directory permissions.
5. **Choosing which local accounts to delete.** The script disables Guest and renames Administrator but deliberately does not delete accounts it does not recognize, because it cannot know which ones the competition expects.
6. **Setting the actual admin password.** Never hardcode that in a script.
7. **Firewall exceptions for your specific services.** The script sets the default posture but does not know which ports your email server, web app, or file shares listen on.

Caveats You Must Read

Test in a snapshot first. If your competition environment allows VM snapshots, take one before running. The script exports `secpol_before.inf`, firewall rules, local users, services, and Defender preferences to the backup folder, but firewall rollback in particular is not a clean one-liner.

Two changes break things most often. Requiring SMB signing can break connections to older NAS devices and some third-party backup appliances. Disabling the Print Spooler breaks printing. Both are correct hardening, but confirm your environment tolerates them.

The reboot matters. Virtualization Based Security, Credential Guard, LSA protection, and the update phase all require a restart to take effect. A reboot mid-competition can cost you points or drop user sessions. Schedule it deliberately.

Do not enable Constrained Language Mode casually. It restricts PowerShell to a safe subset and will break administrative scripts, including ones you may have written for the competition. That is why it is behind a switch rather than on by default.

Verify after the run. Read the log, confirm zero failures, then test that email still flows, users can still authenticate, and your required ports still respond. A server that is secure but not functional scores the same as one that is functional but not secure.